

T0.10 — Whapi webhook signature validation

Diagnostico de solo lectura · Fecha: 2026-05-01 · Modo: solo lectura. · Sin cambios, sin commits, sin push.

Hallazgo clave: Whapi **no usa HMAC signature**. Su modelo de seguridad oficial es **shared secret en custom header** configurado via PATCH /settings. T0.10 implementa ese modelo. Como WHATSAPP_PROVIDER=meta hoy, el PR es preventivo — blinda codigo antes de cualquier cambio futuro de provider, sin riesgo de romper el deploy actual.

1. Flujo actual cuando WHATSAPP_PROVIDER=whapi

```
whapi.cloud -> POST https:///webhook (con headers configurados en su panel)
|
agent/main.py:2154  webhook_handler(request)
|
agent/main.py:771   procesar_webhook(request)
|
proveedor.parsear_webhook(request)
|
+-- WHATSAPP_PROVIDER=whapi -> ProveedorWhapi.parsear_webhook
v
agent/providers/whapi.py:23  parsear_webhook
|
body = await request.json()    <- !SIN VALIDAR FIRMA NI HEADER!
|
procesa mensajes y los devuelve a brain.py / Claude
```

Hoy no se ejecuta porque WHATSAPP_PROVIDER=meta. Pero el endpoint /webhook sigue publico — si alguien cambia la variable a whapi, la linea anterior (request.json()) procesa cualquier payload sin filtro.

2. Que soporta Whapi para validar webhooks

Despues de revisar la documentacion oficial:

Mecanismo	Soportado por Whapi	Detalle
HMAC signature (estilo Stripe / Meta)	No	No hay firma criptografica del body. No hay header X-Whapi-Signature ni equivalente.
Custom headers	Si	Endpoint PATCH /settings con parametro 'headers'. Cualquier header con cualquier valor que el owner configure se incluye en cada webhook saliente desde Whapi.
IP allowlist	No documentado	Whapi no publica rangos oficiales.
Bearer token	Disponible via custom header	El owner puede agregar Authorization: Bearer como custom header.

Modelo de seguridad oficial = shared secret en custom header. El owner configura en el panel Whapi un header arbitrario (ej. X-Webhook-Token: <random>) y el backend valida que el header llegue con el valor esperado.

Esto es **distinto** a T0.2/T0.3/T0.4: no se firma el body, solo se valida un secreto compartido en un header. Replay attacks son posibles si el secret se filtra (no hay timestamp ni nonce signados).

3. Estado actual del codigo

agent/providers/whapi.py:23-32:

```
async def parsear_webhook(self, request: Request) -> list[MensajeEntrante]:
    body = await request.json()
    logger.debug(f"Payload Whapi recibido: {body}")
    # ... procesa directo ...
```

No hay validacion de firma, header ni token. Cualquier POST a /webhook con un JSON tipo {'messages': [{'chat_id': 'X', 'type': 'text', 'text': {'body': 'Hola'}}]} seria procesado como mensaje real si WHATSAPP_PROVIDER=whapi.

Adicionalmente:

- **No hay rate limiting** especifico del endpoint (el rate limit en _dentro_de_limite se aplica por telefono, despues de parsear).
- **No hay replay protection** (no se valida timestamp ni mensaje_id contra ventana temporal — solo deduplicacion por mensaje_id, que el atacante elige).

4. Riesgo real

4.1 Hoy (WHATSAPP_PROVIDER=meta)

- Provider Whapi **no se instancia**. parsear_webhook de Whapi nunca se ejecuta.
- El endpoint /webhook esta corriendo el parser Meta (con HMAC obligatorio desde T0.3).
- **Riesgo activo: cero.**

4.2 Si alguien cambiara WHATSAPP_PROVIDER=whapi

- Inmediatamente todo POST a /webhook se procesaria como Whapi.
- Atacante envia: POST /webhook con {'messages': [{'chat_id': '', 'type': 'text', 'text': {'body': 'Mensaje malicioso'}}]}
- Dona invoca a Claude con ese texto, ejecuta tools, gasta creditos del workspace de la victima, y responde por WhatsApp.
- Bypass del flujo legitimo: **inyeccion de mensajes falsos, gasto de creditos forjado**, posible **acceso a datos** del workspace si el texto induce comandos sensibles.

4.3 Variables legacy en Render

- **WHAPI_TOKEN** — sigue usandose en agent/providers/whapi.py:20, agent/main.py:314,814 y agent/transcriber.py:68. Si bien hoy WHATSAPP_PROVIDER=meta, el token se usa tambien para diagnostico admin (/diagnostico) y para transcribir audios entrantes (legacy de cuando Whapi era default). Mantenerla **si tiene sentido**, pero queda como codigo sin trafico.
- **WHAPI_API_URL** — **no se lee en ningun lugar del codigo** (verificado con grep). Es var dormida 100%. Puede **borrarse de Render** sin impacto. Las URLs de Whapi estan hardcodeadas a https://gate.whapi.cloud/...

5. Variable propuesta para validar firma

Como Whapi no tiene HMAC, el modelo es shared-secret-in-header. Recomendando:

Variable	Funcion	Valor sugerido
WHAPI_WEBHOOK_TOKEN	El secreto que Whapi inyectara como custom header en cada webhook entrante.	string aleatorio largo (secrets.token_urlsafe(32))
WHAPI_WEBHOOK_HEADER (opcional)	Nombre del header donde Whapi envia el token. Default sugerido: X-Webhook-Token. Si lo dejamos default, no necesita ser configurable.	default X-Webhook-Token

El owner debe configurar en el panel de Whapi (via PATCH /settings) que el header X-Webhook-Token con valor WHAPI_WEBHOOK_TOKEN se envíe en cada webhook. Eso conecta los dos lados.

No necesitamos WHAPI_WEBHOOK_SECRET con sufijo _SECRET porque no es secreto criptografico (no se usa para firmar HMAC). Es un shared token.

6. Cambio minimo recomendado

6.1 agent/providers/whapi.py

Agregar al `__init__` (mismo patron que T0.3 Meta):

```
def __init__(self):
    self.token = os.getenv("WHAPI_TOKEN")
    self.url_envio = "https://gate.whapi.cloud/messages/text"
    self.webhook_token = os.getenv("WHAPI_WEBHOOK_TOKEN", "").strip()
    self.webhook_header = os.getenv("WHAPI_WEBHOOK_HEADER", "X-Webhook-Token").strip()

    environment = os.getenv("ENVIRONMENT", "development").lower()
    if environment == "production" and not self.webhook_token:
        raise RuntimeError(
            "[WHAPI] WHAPI_WEBHOOK_TOKEN no configurado en produccion - "
            "los webhooks aceptarían payloads forjados, lo que permite a "
            "un atacante inyectar mensajes WhatsApp falsos. Configura la "
            "variable o cambia WHATSAPP_PROVIDER antes del deploy."
        )
```

Agregar metodo `_verificar_firma` (en realidad 'verificar header'):

```
def _verificar_firma(self, request: Request) -> bool:
    """Valida que el request incluye el custom header con el valor esperado.
    Whapi no firma HMAC; el modelo es shared secret en custom header.
    """
    import hmac
    environment = os.getenv("ENVIRONMENT", "development").lower()
    if not self.webhook_token:
        if environment == "production":
            logger.error("[WHAPI] WHAPI_WEBHOOK_TOKEN no configurado en produccion ...")
            return False
        logger.warning("[WHAPI] sin token, aceptando sin verificar (solo dev/test)")
        return True
    received = request.headers.get(self.webhook_header, "")
    if not hmac.compare_digest(received, self.webhook_token):
        logger.warning(f"[WHAPI] Header {self.webhook_header} ausente o no coincide")
        return False
    return True
```

Llamar al check al inicio de `parsear_webhook`:

```
async def parsear_webhook(self, request: Request) -> list[MensajeEntrante]:
    if not self._verificar_firma(request):
        return [] # Rechazo silencioso - no procesar payload
    body = await request.json()
    # ... resto sin cambios ...
```

6.2 .env.example

Reformular el bloque Whapi (sin valores reales):

```
# == Whapi.cloud (si WHATSAPP_PROVIDER=whapi) ==
WHAPI_TOKEN=
#
# WHAPI_WEBHOOK_TOKEN: OBLIGATORIA en produccion cuando WHATSAPP_PROVIDER=whapi.
# Es un shared secret que Whapi enviara como custom header en cada webhook
# entrante (Whapi no usa HMAC; usa custom headers via PATCH /settings).
# El owner debe configurar en el panel de Whapi:
#   "headers": {"X-Webhook-Token": ""}
# Generar con:
#   python -c "import secrets; print(secrets.token_urlsafe(32))"
# WHAPI_WEBHOOK_TOKEN=
# WHAPI_WEBHOOK_HEADER=X-Webhook-Token # opcional, default X-Webhook-Token
```

6.3 Total del cambio

- `agent/providers/whapi.py`: +25-30 líneas (init check + `_verificar_firma`).
- `.env.example`: +10 líneas documentales.

- tests/test_providers.py: +5-6 tests nuevos.
- PR pequeno, ~40 lineas netas.

7. Conviene fail-fast en startup?

Si, mismo patron que T0.3 Meta:

- El check vive en `__init__` de `ProveedorWhapi`.
- Solo se ejecuta si `obtener_proveedor()` instancia `Whapi` (`WHATSAPP_PROVIDER=whapi`).
- Si esta en meta o twilio, el modulo `Whapi` ni se carga al startup, asi que el check no se dispara.
- **Hoy con `WHATSAPP_PROVIDER=meta` el check no se ejecuta**, asi que no afecta el deploy actual.

Esto es **clave**: T0.10 puede mergearse hoy sin riesgo de romper Render — porque el provider activo sigue siendo Meta. La proteccion queda armada para el dia que se cambie a Whapi.

Defensa en profundidad via `_verificar_firma` tambien se activa solo al primer webhook entrante con `WHATSAPP_PROVIDER=whapi`.

8. Tests a agregar

tests/test_providers.py hoy tiene tests de Meta pero **ninguno de Whapi para webhook validation**. Hay que agregar una clase nueva.

```
class TestWhapiWebhookValidation:
    """T0.10: validacion de header personalizado en webhooks Whapi."""

    def test_dev_sin_token_acepta_payload(self, monkeypatch)
    def test_production_sin_token_levanta_runtime_error(self, monkeypatch)
    def test_production_con_token_no_levanta(self, monkeypatch)
    def test_production_token_valido_acepta(self, monkeypatch)
    def test_production_token_invalido_rechaza(self, monkeypatch)
    def test_production_sin_header_rechaza(self, monkeypatch)
```

Total: 6 tests nuevos + 1 fixture `_make_whapi_text_msg`. Cero tests existentes a actualizar (no habia tests de webhook Whapi).

9. Limpiar o mantener WHAPI_TOKEN y WHAPI_API_URL en Render?

Variable	Estado actual	Recomendacion
WHAPI_TOKEN	Se usa en agent/providers/whapi.py:20, agent/main.py:314,814, agent/transcriber.py:68. Hoy con provider=meta el codigo del provider no se ejecuta, pero agent/transcriber.py llama <code>https://gate.whapi.cloud/media/{audio_id}</code> con WHAPI_TOKEN para descargar audios — verificar si eso sigue corriendo o quedo vestigial al pasar a Meta.	Mantener hasta confirmar que el transcriber usa Meta directo. T0.10 no la toca.
WHAPI_API_URL	No se lee en ningun archivo del codigo. Unicamente aparece como mencion en un script de docs. URLs Whapi estan hardcodeadas.	Borrar de Render con seguridad. Operacion trivial, sin impacto. (Recomendado para una limpieza separada de housekeeping de env vars.)

Importante: este PR T0.10 **no toca WHAPI_TOKEN ni WHAPI_API_URL**. Solo agrega WHAPI_WEBHOOK_TOKEN. La limpieza de WHAPI_API_URL es un housekeeping de env vars aparte (5 segundos en el Render dashboard, sin codigo).

10. Archivos que tocaria

Archivo	Tipo de cambio
agent/providers/whapi.py	Agregar check fail-fast en <code>__init__</code> + agregar metodo <code>_verificar_firma</code> + invocarlo al inicio de parsear_webhook.
.env.example	Reformular bloque Whapi para documentar WHAPI_WEBHOOK_TOKEN (obligatorio en produccion si provider=whapi) y WHAPI_WEBHOOK_HEADER (opcional).
tests/test_providers.py	Agregar TestWhapiWebhookValidation con 6 tests.

Lo que NO toco:

- agent/main.py — sin cambios. Igual que en T0.3, el check vive en `__init__` del provider.

- `agent/providers/whapi.py:enviar_*` — los metodos de envio no requieren cambios.
- `WHAPI_TOKEN` (env var) — sin tocar.
- Comportamiento de parser de mensajes — sin cambios.
- Endpoint `/webhook` (handler en main) — sin cambios.

11. Riesgos de compatibilidad y deploy

Riesgo	Probabilidad	Mitigacion
Render aborta deploy si hoy provider=whapi y falta <code>WHAPI_WEBHOOK_TOKEN</code>	Cero hoy (provider=meta)	El check vive en init del provider Whapi, que no se instancia con provider=meta
Webhook real de Whapi sin custom header configurado en su panel	Solo si en algun momento se cambia a whapi sin coordinar Whapi panel	Documentar en <code>.env.example</code> los pasos de configuracion
Tests existentes rompen	Cero	No hay tests de webhook Whapi previos; solo agregamos.
Replay attacks (Whapi no firma timestamp)	Igual que hoy	Fuera del alcance de T0.10. Mitigable con dedup por <code>mensaje_id</code> (ya hay) y/o agregar TTL de tokens (separado).

Compatibilidad con Whapi:

- El cambio es 100% compatible. Solo agrega validacion; no altera URL ni formato de payload.
- Si Whapi cambia su API en el futuro (improbable a corto plazo), no afecta la firma.

12. Checklist antes de mergear

12.1 Pre-PR

- ☐ Confirmar que `WHATSAPP_PROVIDER=meta` sigue en Render (no se planea cambio inmediato).
- ☐ Decidir si setear `WHAPI_WEBHOOK_TOKEN` ahora en Render preventivamente, o solo cuando se planee rotar provider.
- ☐ Decidir nombre de branch: sugiero **`pr/whapi-webhook-token-obligatorio`**.

12.2 Durante el PR

- ☐ Implementar cambio en `agent/providers/whapi.py`.
- ☐ Reformular `.env.example`.
- ☐ Agregar `TestWhapiWebhookValidation` (6 tests).
- ☐ `pytest -q` localmente — 100% verde.

12.3 Pre-push

- ☐ Diff toca solo los 3 archivos previstos.
- ☐ Ningun secret real en codigo, comentarios ni tests.

[] Commit: *fix(seguridad): WHAPI_WEBHOOK_TOKEN obligatorio en produccion (T0.10)*.

12.4 Post-push, pre-merge

[] Push a origin/pr/whapi-webhook-token-obligatorio.

[] Confirmar que con WHATSAPP_PROVIDER=meta, el deploy de Render no se ve afectado.

12.5 Post-merge

[] Render deploy debe completar sin errores. Provider Meta sigue funcionando.

[] **Si en algun momento futuro** se decide rotar a WHATSAPP_PROVIDER=whapi: setear WHAPI_WEBHOOK_TOKEN en Render, configurar el header en panel Whapi via PATCH /settings, verificar con un mensaje de prueba.

13. Resumen ejecutivo

Aspecto	Detalle
Riesgo que cierra	P0 latente — inyeccion de mensajes WhatsApp falsos via /webhook cuando provider sea Whapi sin token.
Hoy activo	No (provider=meta). PR es preventivo, blindo codigo antes de futuro cambio de provider.
Patron	Identico a T0.3 Meta: check en <code>__init__</code> del provider + defensa en profundidad en <code>_verificar_firma</code> .
Modelo de seguridad	Custom header con shared secret (Whapi no usa HMAC). Distinto a T0.2/T0.3/T0.4.
Variable nueva	WHAPI_WEBHOOK_TOKEN + opcional WHAPI_WEBHOOK_HEADER (default X-Webhook-Token).
Archivos	3 (1 codigo, 1 docs, 1 tests).
Tamano	~40 lineas netas.
Tests	0 actualizados + 6 nuevos.
Riesgo de deploy	Cero hoy — check no se ejecuta con provider=meta.
Variables legacy	WHAPI_API_URL puede borrarse de Render (sin uso en codigo). WHAPI_TOKEN mantener (sigue usandose en transcribir).
Configuracion Whapi panel	Fuera del PR — solo se hace cuando se rote provider a whapi.

Estado: no implementado todavia. Esperando OK del owner con confirmacion de:

- Nombre del header default (X-Webhook-Token) o si preferis otro.
- Si autorizas documentar WHAPI_WEBHOOK_TOKEN aunque hoy no este en Render.
- Si quieres que en este PR tambien agregue una nota en `.env.example` sobre WHAPI_API_URL siendo legacy/borrable.

Sources

- Whapi.cloud API Documentation. <https://whapi.cloud/docs>
- Whapi-Cloud/whatsapp-api-docs — GitHub. <https://github.com/Whapi-Cloud/whatsapp-api-docs>
- Customizable Webhook Headers — Whapi Help Desk. <https://support.whapi.cloud/help-desk/account/customizable-webhook-headers>
- Set the webhook link to the channel — Whapi Help Desk. <https://support.whapi.cloud/help-desk/receiving/webhooks/set-the-webhook-link-to-the-channel>
- Webhooks — Whapi Help Desk. <https://support.whapi.cloud/help-desk/receiving/webhooks>